

Trabalho Prático: Forrageamento e Comunicação em Enxames (Simulação de Formigueiro)

1. Objetivo do Trabalho

O objetivo deste trabalho é implementar um sistema multiagente assíncrono que simula o comportamento de forrageamento (busca por alimento) de um formigueiro. Os alunos deverão modelar as formigas como agentes autônomos e concorrentes em Python (utilizando `asyncio`), aplicando conceitos de busca distribuída, aprendizado com reforço (marcação de caminhos) e protocolos de comunicação.

2. Cenário-Problema e Regras de Negócio

O ambiente é uma grade (grid) ou um grafo onde o "Formigueiro" é o ponto de origem (ex: coordenada $(0, 0)$). O alimento está posicionado em algum nó desconhecido pelas formigas no início da simulação.

Cada formiga (Agente) deve obedecer rigorosamente às seguintes regras:

- **Busca Aleatória e Limite de Energia:** A formiga sai do formigueiro e dá um passo aleatório por vez rumo a nós vizinhos. Ela tem uma restrição máxima de **20 passos** de distanciamento do centro.
- **Fracasso e Reforço Negativo:** Se a formiga atingir o limite de 20 passos sem encontrar o alimento, ela deve retornar ao formigueiro. Ao retornar, ela deve **marcar o caminho percorrido como inválido** (como um feromônio negativo), informando para que as outras formigas (ou ela mesma na próxima tentativa) não utilizem mais essa rota inútil.
- **Sucesso e Compartilhamento de Informação:** Se a formiga encontrar o alimento (em até 20 passos), ela deve retornar imediatamente ao formigueiro. Ao chegar, ela deve **informar às outras formigas o caminho exato** onde o alimento está, permitindo que o enxame explore essa rota de forma direta, sem precisar vagar aleatoriamente.
- **Esgotamento de Recursos:** O alimento possui uma capacidade máxima. Ele **se esgota quando 4 formigas** conseguirem chegar até ele e retornarem ao formigueiro com partes do recurso. Quando isso acontecer, a simulação deve ser encerrada.

3. Tarefas de Implementação para os Alunos

Os alunos podem aproveitar o esqueleto da classe `Agent` e do `executor_prop.py` desenvolvidos anteriormente, adaptando as filas de mensagens (`self.inbox = asyncio.Queue()`) para o novo contexto.

Tarefa A: Modelagem do Ambiente e Movimentação

- Criar o espaço de busca (uma matriz ou dicionário de coordenadas).
- Implementar a corrotina `agir(self)` da formiga para que ela escolha vizinhos aleatórios, guardando o histórico da sua rota atual (ex: uma lista de coordenadas `[(0,0), (0,1), ...]`).

Tarefa B: Lógica de Retorno e "Feromônio" Negativo

- Se o tamanho da rota chegar a 20 passos, a formiga inverte o trajeto, simulando o retorno.
- Ao voltar à origem, a formiga deve registrar globalmente ou transmitir aos vizinhos a sua "lista de passos falhos", atuando como uma restrição. Essa lógica é semelhante à geração de *Nogoods* no algoritmo de *Asynchronous Backtracking* (ABT), onde um agente comunica aos demais uma rota (atribuição) que não leva a um resultado válido.

Tarefa C: Comunicação de Sucesso (Broadcast)

- Quando a coordenada da formiga for igual à do alimento, ela volta à base e envia uma mensagem para a caixa de entrada (`inbox`) de todas as outras formigas ativas contendo a rota do sucesso.
- Isso reflete o algoritmo $LRTA(n)^*$, onde múltiplos agentes exploram um ambiente e compartilham as melhores rotas encontradas (como compartilhar uma tabela de valores atualizada) para acelerar a convergência do sistema para o objetivo.

Tarefa D: O Orquestrador e a Condição de Parada

- O arquivo orquestrador (`executor_prop.py`) não deve ditar a rota das formigas, mas deve monitorar o estado do alimento.
- Deve haver um contador global ou no ambiente. Assim que o contador de formigas bem-sucedidas chegar a 4, o orquestrador aciona `tarefa.cancel()` para todas as corrotinas e imprime um relatório final.

4. Relatório e Análise Crítica

Os alunos deverão entregar o código-fonte em Python e um pequeno relatório respondendo às seguintes questões conceituais baseadas na teoria de sistemas multiagentes:

1. **Exploração vs. Exploração:** No momento em que uma formiga encontra o alimento e avisa as outras, o comportamento do sistema muda de exploração aleatória para a exploração de uma rota conhecida. Explique como o compartilhamento da rota bem-sucedida altera o processo de aprendizado coletivo em comparação com agentes que aprendem de forma totalmente isolada.
2. **Impacto do Reforço Negativo:** Analise os logs do seu programa e descreva o impacto de marcar os caminhos de até 20 passos como "inválidos". Se o mapa for muito grande, essa regra de bloqueio é suficiente para garantir que o alimento seja encontrado rapidamente?
3. **Comunicação Direta vs. Estigmergia:** No trabalho, pedimos que a formiga comunique *diretamente* a rota às outras (passagem de mensagens). Na natureza, as formigas alteram o ambiente (deixando feromônios no chão), uma forma de comunicação indireta. Como você implementaria essa comunicação indireta apenas modificando a classe base dos agentes e o ambiente?